

DSA Revision Notes

Topic 12 — Greedy Algorithms (Complete)

Greedy choice property · Exchange argument proofs · Activity selection
Intervals · Huffman · Scheduling · Jump Game · 25+ C++ examples

12

Sections

25+

C++ Examples

10

Diagrams

28+

LeetCode Problems

TOC Table of Contents**1. Greedy Algorithms — Core Theory**

- Greedy choice property · Optimal substructure · When greedy fails
- Exchange argument proof technique · Greedy vs DP

2. Interval Scheduling & Merging

- Activity Selection (earliest finish) · Merge Intervals
- Insert Interval · Non-overlapping Intervals · Meeting Rooms

3. Interval Covering & Partitioning

- Minimum arrows to burst balloons · Jump Game I & II
- Video Stitching · Minimum number of taps

4. Sorting-Based Greedy

- Two City Scheduling · Assign Cookies · Queue Reconstruction by Height
- Largest Number · Minimum Cost to Hire K Workers

5. Scheduling Problems

- Job Scheduling with Deadlines · Task Scheduler
- Minimum Platforms · Course Schedule III

6. String Greedy

- Remove K Digits · Largest Rectangle via monotonic stack
- Reorganize String · Remove Duplicate Letters

7. Array & Sequence Greedy

- Gas Station · Candy · Trapping Rain Water (greedy)
- Boats to Save People · Wiggle Subsequence

8. Greedy on Graphs — MST

- Kruskal's & Prim's correctness via cut property
- Minimum Spanning Tree problems

9. Huffman Coding

- Optimal prefix-free encoding · Priority queue construction
- Correctness proof via exchange argument

10. Fractional Knapsack

- Value-per-weight greedy · Why 0/1 knapsack needs DP

11. Advanced Greedy Patterns

- IPO (Two Heaps) · Minimum Number of Refueling Stops
- Partition Labels · Hand of Straights

12. Complexity Cheat Sheet & LeetCode Map

1 Greedy Algorithms — Core Theory

1.1 What is a Greedy Algorithm?

A **greedy algorithm** makes the **locally optimal choice** at each step, hoping this leads to a globally optimal solution. Unlike DP, it never reconsiders past choices. Greedy works when the problem has two properties: (1) **Greedy Choice Property** — a globally optimal solution can be reached by making locally optimal choices. (2) **Optimal Substructure** — the optimal solution to the problem contains optimal solutions to subproblems.

When to Use Greedy

GREEDY SIGNALS in a problem statement:

- 'Minimum number of...' or 'Maximum number of...'
- 'Earliest / latest / shortest / largest first'
- Intervals: sort by start/end time, then make decisions left to right
- 'At each step pick the best available option'

GREEDY FAILS when: a local optimum blocks a better global path.

Example: coin change with arbitrary denominations [1,3,4], target=6

Greedy picks $4+1+1=3$ coins, but optimal is $3+3=2$ coins.

-> Use DP instead.

1.2 Exchange Argument — Proving Greedy Correctness

The **exchange argument** is the standard technique to prove a greedy algorithm is optimal: assume an optimal solution OPT that differs from greedy solution G at some step. Show that swapping OPT's choice at that step with G's choice does not worsen the solution. Repeat until OPT and G agree on all choices — G is therefore optimal.

OPT: b placed before a3

After swap: a3 before b



Exchange argument: swap b and a3. If result is no worse, greedy choice (a3 first) is safe.

1.3 Greedy vs Dynamic Programming

Both require optimal substructure. The key difference: greedy makes one choice and recurses, DP evaluates all choices and picks the best. Use greedy when a single local rule is provably optimal.

Problem	Greedy?	Reason
Activity Selection	Yes	Earliest-finish always leaves max future slots

Fractional Knapsack	Yes	Take best ratio; fractions allowed
0/1 Knapsack	No	Taking one item can block a better combination
Shortest Path (pos weights)	Yes	Dijkstra's greedy extraction is provably optimal
Coin Change (arbitrary)	No	Local best denomination may fail globally
Coin Change (canonical)	Yes	US coins: works by mathematical property
Huffman Coding	Yes	Proven optimal by exchange argument
LCS / Edit Distance	No	No greedy choice exists; explore all alignments
Jump Game I (reachable?)	Yes	Track max reach greedily in one pass
Jump Game II (min jumps)	Yes	BFS-level greedy gives optimal jumps
Minimum Spanning Tree	Yes	Cut property guarantees greedy edge selection

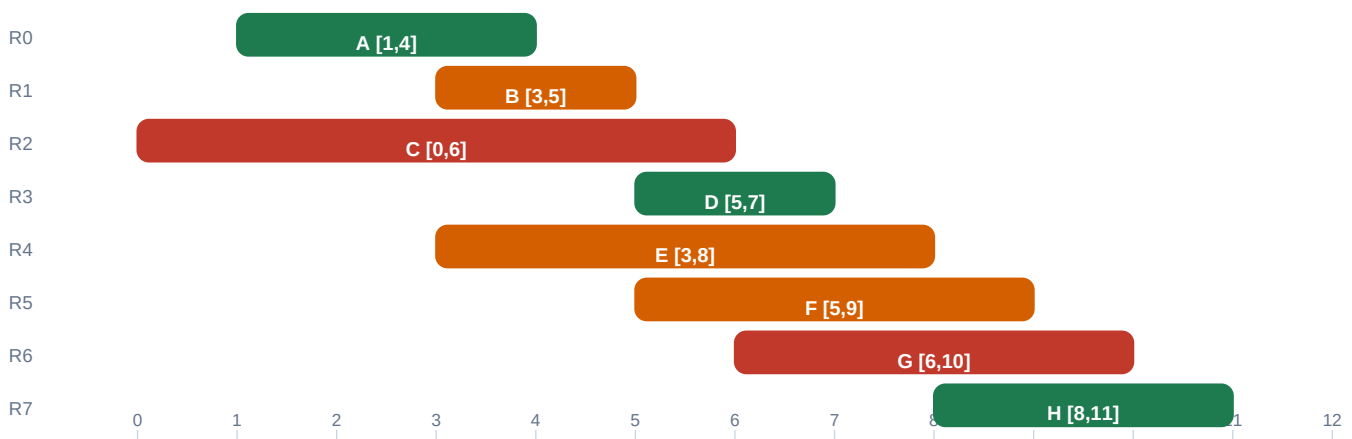
Table 1: Greedy vs DP decision guide

2 Interval Scheduling & Merging

2.1 Activity Selection — Earliest Finish Time

Select maximum number of non-overlapping activities. **Greedy choice:** always pick the activity with the **earliest finish time** that starts after the last selected activity ends. **Proof:** by exchange argument — replacing any activity with the earliest-finish one cannot reduce the count (it finishes earlier, leaving at least as much room for future activities).

Activity Selection: green=selected (A,D,H), orange/red=rejected



C++

```
1 // Activity Selection — Maximum non-overlapping activities
2 int activitySelection(vector<pair<int,int>>& intervals) {
3     // Sort by FINISH time (earliest finish first)
4     sort(intervals.begin(), intervals.end(),
5          [](auto& a, auto& b){ return a.second < b.second; });
6     int count = 1, lastEnd = intervals[0].second;
7     for (int i=1; i<(int)intervals.size(); i++) {
8         if (intervals[i].first >= lastEnd) { // starts after last ends
9             count++;
10            lastEnd = intervals[i].second;
11        }
12    }
13    return count;
14 }
15 // Time: O(n log n) sort + O(n) scan   Space: O(1)
```

2.2 Merge Intervals (LC 56) — Medium

Merge all overlapping intervals. Sort by start time. For each interval, if it overlaps with the last merged interval (start <= lastEnd), extend. Otherwise start a new group.

C++

```

1 // LC 56 - Merge Intervals
2 vector<vector<int>> merge(vector<vector<int>>& intervals) {
3     sort(intervals.begin(), intervals.end()); // sort by start
4     vector<vector<int>> merged;
5     for (auto& iv : intervals) {
6         if (merged.empty() || iv[0] > merged.back()[1]) {
7             merged.push_back(iv); // no overlap: new interval
8         } else {
9             merged.back()[1] = max(merged.back()[1], iv[1]); // overlap: extend
10        }
11    }
12    return merged;
13 }
14 // Time: O(n log n)   Space: O(n)

```

2.3 Insert Interval (LC 57) — Medium

Insert a new interval into a sorted non-overlapping list. Three phases: (1) add all intervals that end before new interval starts, (2) merge all overlapping ones with new interval, (3) add the rest.

C++

```

1 // LC 57 - Insert Interval
2 vector<vector<int>> insert(vector<vector<int>>& intervals,
3                           vector<int>& newInterval) {
4     vector<vector<int>> result;
5     int i=0, n=intervals.size();
6     // Phase 1: add all intervals ending before new interval starts
7     while (i<n && intervals[i][1] < newInterval[0])
8         result.push_back(intervals[i++]);
9     // Phase 2: merge overlapping intervals
10    while (i<n && intervals[i][0] <= newInterval[1]) {
11        newInterval[0] = min(newInterval[0], intervals[i][0]);
12        newInterval[1] = max(newInterval[1], intervals[i][1]);
13        i++;
14    }
15    result.push_back(newInterval);
16    // Phase 3: add remaining intervals
17    while (i<n) result.push_back(intervals[i++]);
18    return result;
19 }
20 // Time: O(n)   Space: O(n) - no sorting needed (already sorted)

```

2.4 Non-overlapping Intervals (LC 435) — Medium

Find minimum number of intervals to remove to make all non-overlapping. **Greedy**: sort by end time, keep intervals that don't overlap (Activity Selection). Answer = total intervals minus maximum kept.

C++

```
1 // LC 435 - Non-overlapping Intervals
2 int eraseOverlapIntervals(vector<vector<int>>& intervals) {
3     sort(intervals.begin(), intervals.end(),
4           [](auto& a, auto& b){ return a[1] < b[1]; }); // sort by end
5     int keep=1, lastEnd=intervals[0][1];
6     for (int i=1; i<(int)intervals.size(); i++) {
7         if (intervals[i][0] >= lastEnd) { // no overlap
8             keep++;
9             lastEnd = intervals[i][1];
10        }
11    }
12    return intervals.size() - keep;
13 }
14 // Time: O(n log n)   Space: O(1)
```

2.5 Meeting Rooms II — Minimum Rooms (LC 253) — Medium

Find minimum number of rooms for all meetings. **Greedy**: sort start times. Use a min-heap of end times to track rooms in use. If earliest ending room finishes before next meeting starts, reuse it; else open a new room.

C++

```
1 // LC 253 - Meeting Rooms II
2 int minMeetingRooms(vector<vector<int>>& intervals) {
3     sort(intervals.begin(), intervals.end()); // sort by start time
4     priority_queue<int, vector<int>, greater<int>> pq; // min-heap of end times
5     for (auto& iv : intervals) {
6         if (!pq.empty() && pq.top() <= iv[0])
7             pq.pop(); // reuse room that ends earliest
8         pq.push(iv[1]); // assign this meeting to a room
9     }
10    return pq.size(); // rooms still in use = total needed
11 }
12 // Time: O(n log n)   Space: O(n)
```

3 Interval Covering & Jump Game

3.1 Minimum Arrows to Burst Balloons (LC 452) — Medium

Each balloon is a horizontal interval. An arrow shot at x bursts all balloons covering x . **Greedy**: sort by end position. Shoot at end of first balloon — this greedily covers as many subsequent balloons as possible. Move to next unpoppped balloon when current arrow misses.

```
C++
1 // LC 452 - Minimum Number of Arrows to Burst Balloons
2 int findMinArrowShots(vector<vector<int>>& points) {
3     sort(points.begin(), points.end(),
4           [](auto& a, auto& b){ return a[1] < b[1]; }); // sort by end
5     int arrows=1, pos=points[0][1];
6     for (int i=1; i<(int)points.size(); i++) {
7         if (points[i][0] > pos) { // current arrow misses this balloon
8             arrows++;
9             pos = points[i][1]; // shoot at end of this balloon
10        }
11    }
12    return arrows;
13 }
14 // Time: O(n log n)   Space: O(1)
```

3.2 Jump Game I (LC 55) — Medium

Can you reach the last index? At each position, track the maximum reachable index. **Greedy**: if current position ever exceeds `maxReach`, we're stuck. Otherwise, update `maxReach = max(maxReach, i + nums[i])`.

```
C++
1 // LC 55 - Jump Game
2 bool canJump(vector<int>& nums) {
3     int maxReach = 0;
4     for (int i=0; i<(int)nums.size(); i++) {
5         if (i > maxReach) return false; // stuck - can't reach position i
6         maxReach = max(maxReach, i + nums[i]);
7     }
8     return true;
9 }
10 // Time: O(n)   Space: O(1)
```

3.3 Jump Game II (LC 45) — Medium

Find minimum number of jumps to reach the last index. **BFS-level greedy**: treat each jump as a BFS level. Track current level's rightmost reach. When we step past the current farthest, increment jumps and extend the boundary.

C++

```

1 // LC 45 - Jump Game II (minimum jumps)
2 int jump(vector<int>& nums) {
3     int jumps=0, curEnd=0, farthest=0;
4     for (int i=0; i<(int)nums.size()-1; i++) {
5         farthest = max(farthest, i + nums[i]); // max reach from current level
6         if (i == curEnd) { // reached end of current jump level
7             jumps++;
8             curEnd = farthest; // extend to next level
9         }
10    }
11    return jumps;
12 }
13 // Time: O(n)   Space: O(1)
14 // Analogy: BFS where each 'level' is one jump.
15 // We don't need to know WHICH cell to jump to - just the farthest reachable.

```

3.4 Video Stitching (LC 1024) — Medium

Given clips [start,end], cover [0,T] with minimum clips. **Greedy:** at each step, among all clips that start $\leq$ current reach, pick the one with the maximum end time (extends coverage the most).

C++

```

1 // LC 1024 - Video Stitching
2 int videoStitching(vector<vector<int>>& clips, int time) {
3     // maxEnd[i] = farthest end of any clip starting at i
4     vector<int> maxEnd(time+1, 0);
5     for (auto& c:clips)
6         if (c[0]<=time) maxEnd[c[0]]=max(maxEnd[c[0]],c[1]);
7     int count=0, curEnd=0, farthest=0;
8     for (int i=0; i<time; i++) {
9         farthest=max(farthest,maxEnd[i]);
10        if (i==curEnd) {
11            if (farthest==curEnd) return -1; // no progress possible
12            count++;
13            curEnd=farthest;
14            if (curEnd>=time) break;
15        }
16    }
17    return count;
18 }
19 // Time: O(n + T)   Space: O(T)

```

4 Sorting-Based Greedy

4.1 Two City Scheduling (LC 1029) — Medium

Send n people to city A and n people to city B minimising total cost. **Greedy:** compute cost difference ($\text{costA} - \text{costB}$) for each person. Sort by this difference. First n people (biggest preference for A) go to A; rest go to B.

```
C++
1 // LC 1029 - Two City Scheduling
2 int twoCitySchedCost(vector<vector<int>>& costs) {
3     // Sort by (costA - costB): send people cheapest-to-divert-to-A first
4     sort(costs.begin(), costs.end(),
5         [](auto& a, auto& b){
6             return (a[0]-a[1]) < (b[0]-b[1]);
7         });
8     int n=costs.size()/2, total=0;
9     for (int i=0;i<n;i++) total+=costs[i][0]; // first n go to A
10    for (int i=n;i<2*n;i++) total+=costs[i][1]; // rest go to B
11    return total;
12 }
13 // Time: O(n log n) Space: O(1)
14 // Proof: imagine sending all to A first (cost = sum of costA).
15 // Switching person i from A to B saves (costA[i] - costB[i]).
16 // Sort by this savings and switch the n most beneficial.
```

4.2 Queue Reconstruction by Height (LC 406) — Medium

Each person $[h, k]$ has height h and k people taller in front. **Greedy:** sort by height descending (taller first), then by k ascending. Insert each person at position k — since all people inserted before are taller, the k -count is automatically satisfied.

```
C++
1 // LC 406 - Queue Reconstruction by Height
2 vector<vector<int>> reconstructQueue(vector<vector<int>>& people) {
3     // Sort: taller first, then by k ascending
4     sort(people.begin(), people.end(),
5         [](auto& a, auto& b){
6             return a[0]!=b[0] ? a[0]>b[0] : a[1]<b[1];
7         });
8     vector<vector<int>> result;
9     for (auto& p : people) {
10        // Insert at position p[1]
11        result.insert(result.begin()+p[1], p);
12    }
13    return result;
14 }
15 // Time: O(n^2) due to insert Space: O(n)
16 // O(n log n) possible with BIT/Fenwick tree
```

4.3 Largest Number (LC 179) — Medium

Arrange numbers to form the largest number. **Greedy comparator:** for two numbers a and b , compare string ab vs ba . If $ab > ba$, a should come first. Sort using this comparator.

C++

```
1 // LC 179 - Largest Number
2 string largestNumber(vector<int>& nums) {
3     vector<string> strs;
4     for (int x:nums) strs.push_back(to_string(x));
5     sort(strs.begin(), strs.end(),
6         [](const string& a, const string& b){
7             return a+b > b+a; // greedy: which concatenation is larger?
8         });
9     if (strs[0]=="0") return "0"; // all zeros
10    string result="";
11    for (auto& s:strs) result+=s;
12    return result;
13 }
14 // Time: O(n log n * L) where L=max digits Space: O(n)
15 // Correctness: comparator a+b > b+a defines a total order (transitivity holds)
```

5 Scheduling Problems

5.1 Task Scheduler (LC 621) — Medium

Schedule tasks with cooldown n between same tasks. Minimise total time. **Greedy:** always schedule the most frequent remaining task. The mathematical formula: $\max(n_tasks, (maxFreq-1)*(n+1) + countOfMaxFreq)$.

```
C++
1 // LC 621 - Task Scheduler
2 int leastInterval(vector<char>& tasks, int n) {
3     vector<int> freq(26,0);
4     for (char c:tasks) freq[c-'A']++;
5     int maxFreq = *max_element(freq.begin(),freq.end());
6     // Count how many tasks share the maximum frequency
7     int maxCount = count(freq.begin(),freq.end(),maxFreq);
8     // Formula: arrange maxFreq-1 blocks of (n+1) + tail
9     int frames = (maxFreq-1)*(n+1) + maxCount;
10    // If we have enough tasks to fill all slots, no idle needed
11    return max((int)tasks.size(), frames);
12 }
13 // Time: O(n) Space: O(1)
14 // Example: tasks=[A,A,A,B,B,B], n=2
15 // maxFreq=3, maxCount=2: frames=(3-1)*(2+1)+2=8
16 // Layout: A B _ A B _ A B (8 intervals)
```

5.2 Job Scheduling with Deadlines — Maximise Profit

Each job has a deadline and profit. Do each job in one unit. **Greedy:** sort by profit descending. For each job, schedule it as late as possible before its deadline (greedy: delay to leave earlier slots for other jobs).

```
C++
1 // Job Scheduling: each job (profit, deadline), 1 unit each
2 int jobScheduling(vector<pair<int,int>>& jobs) {
3     // Sort by profit descending
4     sort(jobs.begin(), jobs.end(),
5          [](auto& a, auto& b){ return a.first>b.first; });
6     int maxDL = 0;
7     for (auto& j:jobs) maxDL=max(maxDL,j.second);
8     vector<int> slot(maxDL+1, -1); // slot[t] = job assigned at time t
9     int totalProfit = 0;
10    for (auto& [profit,dl]:jobs) {
11        // Try to schedule as late as possible before deadline
12        for (int t=min(dl,maxDL); t>=1; t--) {
13            if (slot[t]==-1) {
14                slot[t]=profit;
15                totalProfit+=profit;
16                break;
17            }
18        }
19    }
20    return totalProfit;
21 }
22 // Time: O(n^2) naive - O(n log n) with Union-Find for slot lookup
```

5.3 Course Schedule III (LC 630) — Hard

Maximise number of courses taken. Each course has duration and deadline. **Greedy:** sort by deadline. Keep a max-heap of durations taken so far. If adding a course would exceed its deadline, replace the longest taken course if it is longer than the current one (saves time without reducing count).

C++

```
1 // LC 630 - Course Schedule III
2 int scheduleCourse(vector<vector<int>>& courses) {
3     sort(courses.begin(), courses.end(),
4         [](auto& a, auto& b) { return a[1] < b[1]; }); // sort by deadline
5     priority_queue<int> pq; // max-heap of durations taken
6     int time = 0;
7     for (auto& c : courses) {
8         int dur = c[0], dl = c[1];
9         if (time + dur <= dl) {
10             time += dur; pq.push(dur); // can take this course
11         } else if (!pq.empty() && pq.top() > dur) {
12             // Replace longest course with current (shorter = saves time)
13             time -= pq.top(); pq.pop();
14             time += dur; pq.push(dur);
15         }
16     }
17     return pq.size();
18 }
19 // Time: O(n log n)   Space: O(n)
```

6 String Greedy

6.1 Remove K Digits (LC 402) — Medium

Remove k digits from a number string to get the smallest possible number. **Greedy:** using a monotonic increasing stack, whenever a digit is larger than the next digit, remove it (costs one of our k removals). This ensures the most significant digits are as small as possible.

```
C++
1 // LC 402 - Remove K Digits
2 string removeKdigits(string num, int k) {
3     string st=""; // use string as monotonic stack
4     for (char c:num) {
5         // Remove larger digits from top while we still have k removals left
6         while (k>0 && !st.empty() && st.back()>c) {
7             st.pop_back(); k--;
8         }
9         st+=c;
10    }
11    // If k removals remain, remove from end (largest remaining digits)
12    st.resize(st.size()-k);
13    // Remove leading zeros
14    int start=0;
15    while (start<(int)st.size()-1 && st[start]=='0') start++;
16    return st.substr(start);
17 }
18 // Time: O(n)   Space: O(n)
```

6.2 Partition Labels (LC 763) — Medium

Partition string so each letter appears in at most one part. Maximise number of parts. **Greedy:** for each character, track its last occurrence. Scan left to right, extending the current partition's end to max(end, lastOccurrence[c]). When we reach the end of the current partition, record it and start a new one.

```
C++
1 // LC 763 - Partition Labels
2 vector<int> partitionLabels(string s) {
3     // Step 1: find last occurrence of each character
4     vector<int> last(26,0);
5     for (int i=0;i<(int)s.size();i++) last[s[i]-'a']=i;
6     // Step 2: greedy scan
7     vector<int> result;
8     int start=0, end=0;
9     for (int i=0;i<(int)s.size();i++) {
10        end=max(end, last[s[i]-'a']); // extend partition to include last pos
11        if (i==end) { // reached end of current partition
12            result.push_back(end-start+1);
13            start=end+1;
14        }
15    }
16    return result;
17 }
18 // Time: O(n)   Space: O(1) - 26-char alphabet
```

6.3 Reorganize String (LC 767) — Medium

Rearrange so no two adjacent characters are the same. **Greedy:** always place the most frequent remaining character that differs from the previous. Use a max-heap. If max frequency $> (n+1)/2$, it's impossible.

C++

```
1 // LC 767 - Reorganize String
2 string reorganizeString(string s) {
3     vector<int> freq(26,0);
4     for (char c:s) freq[c-'a']++;
5     // Max-heap of {frequency, character}
6     priority_queue<pair<int, char>> pq;
7     for (int i=0; i<26; i++) if(freq[i]) pq.push({freq[i], 'a'+i});
8     string result="";
9     while (pq.size()>=2) {
10         auto [f1, c1]=pq.top(); pq.pop();
11         auto [f2, c2]=pq.top(); pq.pop();
12         result+=c1; result+=c2;
13         if(f1-1>0) pq.push({f1-1, c1});
14         if(f2-1>0) pq.push({f2-1, c2});
15     }
16     if (!pq.empty()) {
17         if(pq.top().first>1) return ""; // impossible
18         result+=pq.top().second;
19     }
20     return result;
21 }
22 // Time: O(n log k) k=distinct chars Space: O(k)
```

7 Array & Sequence Greedy

7.1 Gas Station (LC 134) — Medium

Find starting gas station to complete a circular route. **Key insight:** if total gas $\geq$ total cost, a solution always exists.

Greedy: track running tank. When it goes negative, the start must be after current position.

```
C++
1 // LC 134 - Gas Station
2 int canCompleteCircuit(vector<int>& gas, vector<int>& cost) {
3     int total=0, tank=0, start=0;
4     for (int i=0; i<(int)gas.size(); i++) {
5         int diff = gas[i]-cost[i];
6         total+=diff;
7         tank +=diff;
8         if (tank<0) { // can't start before or at i
9             start=i+1; // try starting from next station
10            tank=0; // reset tank
11        }
12    }
13    return total>=0 ? start : -1;
14 }
15 // Time: O(n) Space: O(1)
16 // Why is 'start' the answer? If total>=0, there must be a valid start.
17 // The last segment that made tank negative disqualifies all earlier starts.
```

7.2 Candy (LC 135) — Hard

Give children candy: (1) each child gets 1+ candy; (2) child with higher rating than neighbour gets more. **Greedy — two passes:** left-to-right satisfies left neighbours; right-to-left satisfies right neighbours. Take max of both passes for each child.

```
C++
1 // LC 135 - Candy
2 int candy(vector<int>& ratings) {
3     int n=ratings.size();
4     vector<int> candies(n,1);
5     // Pass 1: left to right - if ratings[i] > ratings[i-1], give more than left
6     for (int i=1; i<n; i++)
7         if (ratings[i]>ratings[i-1]) candies[i]=candies[i-1]+1;
8     // Pass 2: right to left - if ratings[i] > ratings[i+1], ensure more than right
9     for (int i=n-2; i>=0; i--)
10        if (ratings[i]>ratings[i+1])
11            candies[i]=max(candies[i], candies[i+1]+1);
12    return accumulate(candies.begin(), candies.end(), 0);
13 }
14 // Time: O(n) Space: O(n)
```

7.3 Boats to Save People (LC 881) — Medium

Each boat holds at most 2 people with weight limit. Minimise boats. **Greedy:** sort people by weight. Use two pointers — pair heaviest with lightest if possible. If they fit together, both board; otherwise heaviest boards alone.

C++

```
1 // LC 881 - Boats to Save People
2 int numRescueBoats(vector<int>& people, int limit) {
3     sort(people.begin(), people.end());
4     int lo=0, hi=people.size()-1, boats=0;
5     while (lo<=hi) {
6         if (people[lo]+people[hi]<=limit) lo++; // lightest pairs with heaviest
7         hi--; // heaviest always boards (paired or alone)
8         boats++;
9     }
10    return boats;
11 }
12 // Time: O(n log n)   Space: O(1)
```

7.4 Wiggle Subsequence (LC 376) — Medium

Find longest wiggle subsequence (alternating up/down). **Greedy**: count direction changes. A peak or valley always contributes.

C++

```
1 // LC 376 - Wiggle Subsequence
2 int wiggleMaxLength(vector<int>& nums) {
3     if (nums.size()<2) return nums.size();
4     int up=1, down=1;
5     for (int i=1;i<(int)nums.size();i++) {
6         if (nums[i]>nums[i-1]) up=down+1; // rising: extend down sequence
7         else if (nums[i]<nums[i-1]) down=up+1; // falling: extend up sequence
8     }
9     return max(up,down);
10 }
11 // Time: O(n)   Space: O(1)
```

8 Greedy on Graphs — Minimum Spanning Tree

8.1 Cut Property — Why MST Greedy is Correct

A **cut** partitions the vertices into two disjoint sets (S , $V-S$). The **cut property**: for any cut, the minimum-weight edge crossing the cut belongs to SOME minimum spanning tree. Both Kruskal's and Prim's exploit this: they always add the cheapest edge that connects the current MST fragment to an unvisited vertex.

MST Greedy Correctness

Cut Property: if e is the min-weight edge crossing any cut (S , $V-S$), then e is in some MST.

Cycle Property: if e is the max-weight edge in any cycle, then e is not in any MST.

Kruskal: sort edges by weight $\rightarrow$ add if it doesn't create a cycle (Union-Find) $\rightarrow O(E \log E)$

Prim: start from any vertex $\rightarrow$ always add cheapest edge to unvisited vertex $\rightarrow O((V+E) \log V)$

Both are greedy and correct by the cut property.

Key difference: Kruskal = edge-focused (global sort). Prim = vertex-focused (grow tree).

C++

```

1 // Kruskal's MST - O(E log E) using Union-Find
2 int kruskalMST(int n, vector<tuple<int,int,int>>& edges) {
3     sort(edges.begin(), edges.end()); // sort by weight
4     vector<int> par(n); iota(par.begin(), par.end(), 0);
5     vector<int> rnk(n, 0);
6     function<int(int)> find = [&](int x) -> int {
7         return par[x] == x ? x : par[x] = find(par[x]);
8     };
9     auto unite = [&](int x, int y) -> bool {
10         x = find(x); y = find(y); if (x == y) return false;
11         if (rnk[x] < rnk[y]) swap(x, y);
12         par[y] = x; if (rnk[x] == rnk[y]) rnk[x]++; return true;
13     };
14     int mstCost = 0, edges_used = 0;
15     for (auto& [w, u, v] : edges) {
16         if (unite(u, v)) { mstCost += w; edges_used++;
17             if (edges_used == n - 1) break; }
18     }
19     return edges_used == n - 1 ? mstCost : -1;
20 }
21 // Time: O(E log E)   Space: O(V)

```

9 Huffman Coding — Optimal Prefix-Free Encoding

9.1 Problem & Greedy Strategy

Huffman coding assigns shorter bit codes to more frequent characters. The result is an optimal prefix-free (no code is prefix of another) binary code. **Greedy strategy:** always merge the two least-frequent symbols into a combined node. This bottom-up tree construction is provably optimal by exchange argument.

Huffman Algorithm

Algorithm: use a min-heap of (frequency, node).

1. Insert all characters with their frequencies.
2. While heap has more than 1 node:
 - a. Extract two minimum-frequency nodes L and R.
 - b. Create parent node with frequency = freq(L) + freq(R).
 - c. Insert parent back into heap.
3. The remaining node is the root of the Huffman tree.
4. Assign 0 for left branches, 1 for right. Path from root = code for each character.

Optimality proof: exchange argument shows merging two least-freq nodes first is always safe.

C++

```

1 // Huffman Coding - minimum total encoding cost
2 struct HNode {
3     int freq;
4     char ch;
5     HNode *left, *right;
6     HNode(int f, char c='#'):freq(f),ch(c),left(nullptr),right(nullptr){}
7 };
8 struct Cmp { bool operator()(HNode* a,HNode* b){return a->freq>b->freq;} };
9
10 long long huffmanCost(vector<pair<char,int>>& charFreq) {
11     priority_queue<HNode*,vector<HNode*>,Cmp> pq;
12     for (auto&[c,f]:charFreq) pq.push(new HNode(f,c));
13     long long totalCost=0;
14     while (pq.size()>1) {
15         HNode* L=pq.top(); pq.pop();
16         HNode* R=pq.top(); pq.pop();
17         int combined=L->freq+R->freq;
18         totalCost+=combined; // each merge costs combined frequency
19         HNode* parent=new HNode(combined);
20         parent->left=L; parent->right=R;
21         pq.push(parent);
22     }
23     return totalCost;
24 }
25 // Time: O(n log n) Space: O(n)
26 // totalCost = sum of all internal node frequencies = weighted path length

```

10 Fractional Knapsack

10.1 Fractional Knapsack — Value/Weight Greedy

Items can be broken into fractions. Fill capacity with the highest value-per-weight items first. **Greedy:** sort by value/weight ratio descending. Take as much of each item as possible. This is optimal because fractional inclusion means we never 'waste' capacity. **Why greedy fails for 0/1 knapsack:** without fractions, taking a high-ratio small item can block a better combination of larger items.

C++

```

1  // Fractional Knapsack
2  double fractionalKnapsack(vector<int>& wt, vector<int>& val, int W) {
3      int n=wt.size();
4      vector<int> idx(n); iota(idx.begin(),idx.end(),0);
5      // Sort by value/weight ratio descending
6      sort(idx.begin(),idx.end(),[&](int a,int b){
7          return (double)val[a]/wt[a] > (double)val[b]/wt[b];
8      });
9      double totalVal=0.0;
10     int capacity=W;
11     for (int i:idx) {
12         if (wt[i]<=capacity) {
13             totalVal+=val[i]; // take entire item
14             capacity-=wt[i];
15         } else {
16             totalVal+=val[i]*(double)capacity/wt[i]; // take fraction
17             break; // capacity exhausted
18         }
19     }
20     return totalVal;
21 }
22 // Time: O(n log n) Space: O(n)
23
24 // Compare with 0/1 knapsack counterexample:
25 // Items: {wt=10, val=60}, {wt=20, val=100}, {wt=30, val=120}, W=50
26 // Greedy ratios: 6, 5, 4 -> take item1(60)+item2(100)+10/30*item3(40)=200
27 // But 0/1 optimal: item2+item3=220 -> greedy gives WRONG answer for 0/1

```

11 Advanced Greedy Patterns

11.1 IPO — Maximise Capital (LC 502) — Hard

Pick at most k projects (one at a time) to maximise capital. **Two-heap greedy:** min-heap by capital (locked projects), max-heap by profit (available). At each step: unlock all affordable projects, then pick the most profitable one.

C++

```

1 // LC 502 - IPO (Two Heap Greedy)
2 int findMaximizedCapital(int k, int w,
3                           vector<int>& profits, vector<int>& capital) {
4     int n=profits.size();
5     // Min-heap by required capital
6     priority_queue<pair<int,int>, vector<pair<int,int>>,
7                   greater<>> locked;
8     // Max-heap by profit (available projects)
9     priority_queue<int> available;
10    for (int i=0; i<n; i++) locked.push({capital[i], profits[i]});
11    for (int i=0; i<k; i++) {
12        // Unlock all projects we can afford
13        while (!locked.empty() && locked.top().first <= w) {
14            available.push(locked.top().second);
15            locked.pop();
16        }
17        if (available.empty()) break; // no affordable project
18        w+=available.top(); available.pop(); // take most profitable
19    }
20    return w;
21 }
22 // Time: O(n log n + k log n)   Space: O(n)

```

11.2 Minimum Number of Refueling Stops (LC 871) — Hard

Drive from 0 to target. Refuel at stations. Find minimum stops. **Greedy:** when we run out of fuel, retroactively refuel at the largest-capacity station we passed. Use a max-heap of passed station fuels.

C++

```

1 // LC 871 - Minimum Number of Refueling Stops
2 int minRefuelStops(int target, int startFuel,
3                   vector<vector<int>>& stations) {
4     priority_queue<int> pq; // max-heap of passed station fuels
5     int fuel=startFuel, stops=0, prev=0;
6     stations.push_back({target, 0}); // add target as final station
7     for (auto& s: stations) {
8         fuel -= (s[0] - prev); // fuel consumed to reach this station
9         prev = s[0];
10        while (fuel < 0 && !pq.empty()) { // out of fuel - retroactively refuel
11            fuel += pq.top(); pq.pop(); stops++;
12        }
13        if (fuel < 0) return -1; // can't reach even with all stops
14        pq.push(s[1]); // add this station's fuel to candidates
15    }
16    return stops;
17 }
18 // Time: O(n log n)   Space: O(n)

```

11.3 Hand of Straights (LC 846) — Medium

Divide hand into groups of consecutive cards of size `groupSize`. **Greedy**: always start a new group with the smallest available card. Use a sorted map to track counts.

```
C++  
1 // LC 846 - Hand of Straights  
2 bool isNStraightHand(vector<int>& hand, int groupSize) {  
3     if (hand.size()%groupSize!=0) return false;  
4     map<int,int> cnt;  
5     for (int x:hand) cnt[x]++;  
6     for (auto&[start,c]:cnt) {  
7         if (c==0) continue;  
8         // Try to form 'c' groups starting at 'start'  
9         for (int i=0;i<groupSize;i++) {  
10            if (cnt[start+i]<c) return false;  
11            cnt[start+i]-=c;  
12        }  
13    }  
14    return true;  
15 }  
16 // Time: O(n log n)   Space: O(n)
```

12

Complexity Cheat Sheet & LeetCode Map

12.1 Greedy Algorithm Complexity Summary

Problem / Pattern	Sort By	Data Struct	Time	Key Greedy Choice
Activity Selection	End time	—	$O(n \log n)$	Pick earliest-finish non-overlapping
Merge Intervals	Start time	—	$O(n \log n)$	Extend or start new interval
Non-overlapping (min remove)	End time	—	$O(n \log n)$	Keep maximum compatible set
Meeting Rooms II (min rooms)	Start time	Min-heap	$O(n \log n)$	Reuse earliest-ending room
Burst Balloons (min arrows)	End position	—	$O(n \log n)$	Shoot at end of each group
Jump Game II (min jumps)	—	—	$O(n)$	Extend reach level by level
Task Scheduler	Frequency	Max-heap	$O(n)$	Math formula or simulation
Job with Deadline (max profit)	Profit desc	Slots arr	$O(n^2)$	Schedule latest before deadline
Course Schedule III	Deadline	Max-heap	$O(n \log n)$	Replace longest if shorter fits
Gas Station	—	—	$O(n)$	Reset start when tank < 0
Candy (two-pass)	—	—	$O(n)$	L-to-R then R-to-L passes
Two City Scheduling	costA-costB	—	$O(n \log n)$	Send n cheapest-to-divert to A
Queue Reconstruction	Height desc	—	$O(n^2)$	Insert at position k
Largest Number	Custom: $a+b > b+a$	—	$O(n \log n)$	Transitive comparator
Huffman Coding	Frequency	Min-heap	$O(n \log n)$	Merge two least frequent
Fractional Knapsack	Value/weight ratio	—	$O(n \log n)$	Take highest ratio first
IPO (max capital)	Capital	Two heaps	$O(n \log n)$	Unlock affordable, pick max profit
Min Refueling Stops	Position	Max-heap	$O(n \log n)$	Retroactively use largest passed
Partition Labels	—	—	$O(n)$	Extend cut to last occurrence

Table 2: Greedy algorithm patterns with sorting keys and data structures

12.2 Pattern Recognition Guide

C++

```

1  /*
2  * PROBLEM SIGNAL                                -> GREEDY PATTERN
3  * -----
4  * Max non-overlapping intervals                 -> Sort by END time, scan
5  * Min intervals to remove                     -> Sort by END, count kept
6  * Merge overlapping intervals                  -> Sort by START, extend
7  * Min rooms / machines needed                 -> Sort START, min-heap of END times
8  * Cover [0,T] with minimum segments           -> Jump Game II style (level BFS)
9  * Min arrows to burst balloons                -> Sort by END, shoot at END
10 * Min jumps to reach end                      -> Track farthest reach, count levels
11 * Max profit from projects with capital req   -> Two heaps (locked/available)
12 * Optimal binary encoding                     -> Huffman (min-heap, merge two least)
13 * Can you complete a circular route?         -> Gas Station: track net gain
14 * Remove digits to minimise number            -> Monotonic increasing stack
15 * Arrange numbers to form max value           -> Custom comparator a+b > b+a
16 * Schedule tasks with cooldown               -> Frequency formula or simulation
17 * Divide items into groups of consecutive     -> Sort + map, greedily form groups
18 * Distribute resources with min cost          -> Sort, take greedily with two pointers
19 * Must prove greedy: use EXCHANGE ARGUMENT   -> Swap adjacent choices, show no worse
20 */

```

12.3 LeetCode Problem Map

#	Problem	Pattern	Diff
45	Jump Game II	BFS-level greedy	Medium
55	Jump Game	Max reach tracking	Medium
56	Merge Intervals	Sort by start, extend	Medium
57	Insert Interval	Three-phase scan	Medium
134	Gas Station	Net gain, reset start	Medium
135	Candy	Two-pass greedy	Hard
179	Largest Number	Custom comparator sort	Medium
253	Meeting Rooms II	Sort + min-heap	Medium
376	Wiggle Subsequence	Count direction changes	Medium
392	Is Subsequence	Two-pointer greedy	Easy
402	Remove K Digits	Monotonic stack greedy	Medium
406	Queue Reconstruction by Height	Sort + insert at k	Medium
435	Non-overlapping Intervals	Sort by end, max compatible	Medium
452	Minimum Arrows to Burst Balloons	Sort by end, shoot at end	Medium
502	IPO	Two heaps greedy	Hard
621	Task Scheduler	Frequency + formula	Medium
630	Course Schedule III	Sort deadline + max-heap	Hard

646	Maximum Length of Pair Chain	Activity selection	Medium
714	Best Time to Buy/Sell Stock with Fee	State machine greedy	Medium
763	Partition Labels	Last occurrence scan	Medium
767	Reorganize String	Max-heap frequency	Medium
846	Hand of Straights	Sorted map groups	Medium
860	Lemonade Change	Greedy cash tracking	Easy
871	Min Refueling Stops	Retroactive max-heap	Hard
881	Boats to Save People	Two-pointer greedy	Medium
1029	Two City Scheduling	Sort by cost diff	Medium
1024	Video Stitching	Jump Game II style	Medium
1642	Furthest Building You Can Reach	Min-heap + greedy ladders	Medium
2263	Make Array Beautiful	Greedy scan	Medium

Table 3: 28 LeetCode problems — Greedy Algorithms

Golden Rules — Greedy Algorithms

1. To PROVE greedy: use exchange argument — show swapping greedy choice never worsens the result.
2. To DISPROVE greedy: find one counterexample where local best blocks global optimum -> use DP.
3. Intervals: sort by END for selection/arrows/non-overlap. Sort by START for merging.
4. Jump Game: track maxReach greedily. Jump Game II: BFS-level — count jumps when $i == \text{curEnd}$.
5. Two-heap pattern: min-heap of locked resources + max-heap of available -> $O(\log n)$ per step.
6. Gas Station: if total gas $\geq$ total cost, solution exists. Reset start whenever tank < 0 .
7. Huffman: always merge two least frequent. Optimality follows from exchange argument.
8. Fractional Knapsack: sort by value/weight descending. Take greedily. Never works for 0/1.
9. Custom comparator (Largest Number): $a+b > b+a$. Must verify transitivity for sort correctness.
10. Meeting Rooms: min rooms = max overlapping at any point = PQ size after greedy assignment.